Lesson 15 — Movies Analysis Dashboard

1) Data cleaning & column types (Power Query recommended)

Goal: Ensure `Budget` and `Box Office` are numeric, `Release Date` is Date, and missing values handled.

Recommended approach: *fix in Power Query* (Home → Transform data), not with DAX. Steps:

1. Open Query Editor. Select `Movies` query.
2. Set column types:
   - `Release Date` → Date
   - `Budget`, `Box Office` → Decimal Number (or Whole Number as appropriate)
   - `Run Time`, `Oscar Wins`, `Nominations` → Whole Number
3. Handle missing values:
   - Quick option: Right-click column → Replace Values → Replace `null` with `0` (if business rule says unknown = 0).
   - Better option (recommended): Replace with median or contextual value:
     - Home → Group By → group nothing and create operation `MedianBudget = Median([Budget])` — if UI supports median.
     - Or compute median in a separate query and then Fill Down / Merge to replace nulls with median for rows missing value.
4. Example M (replace null with 0 for both columns):

```
#"Replaced Nulls" = Table.ReplaceValue(#"PreviousStep", null, 0,
Replacer.ReplaceValue, {"Budget","Box Office"})
```

Why Power Query? It changes the stored data (model size and downstream calculations are simpler). Use DAX only for computed results that must respect report filters.

---

2) Date table — build and mark as Date Table

Why: Time intelligence DAX functions require a contiguous, marked Date table (one row per date) and proper relationship to `Movies[Release Date]`.

Two recommended options:

A — CALENDARAUTO + ADDCOLUMNS (quick)

```
DateTable =
ADDCOLUMNS(
    CALENDARAUTO(),
    "Year", YEAR([Date]),
    "Month", MONTH([Date]),
    "MonthName", FORMAT([Date],"MMMM"),
    "MonthKey", YEAR([Date])*100 + MONTH([Date]),
    "Quarter", "Q" & FORMAT([Date],"Q")
)
```

B — Explicit calendar limited to movie range (safer)

```
DateTable =
VAR _Min = MINX(ALL('Movies'), 'Movies'[Release Date])
VAR _Max = MAXX(ALL('Movies'), 'Movies'[Release Date])
RETURN
ADDCOLUMNS(
    CALENDAR(_Min, _Max),
    "Year", YEAR([Date]),
    "Month", MONTH([Date]),
    "MonthName", FORMAT([Date],"MMMM"),
    "MonthKey", YEAR([Date])*100 + MONTH([Date]),
    "Quarter", "Q" & FORMAT([Date],"Q")
)
```

After creating, go Model → Mark as Date Table → choose `DateTable[Date]`. Then ensure relationship: `Movies[Release Date] (many) -> DateTable[Date] (one)`, single-directional filtering (recommended).

---

3) Calculated columns

Use calculated **columns** for row-level static values (categories, profit as stored per row). They are computed at model-refresh and stored in memory.

A — Profit (per movie)

```
Profit =
IF(
    ISBLANK( Movies[Box Office] ) || ISBLANK( Movies[Budget] ),
    BLANK(),
    Movies[Box Office] - Movies[Budget]
)
```

Notes: If you prefer zero instead of BLANK when budgets absent, replace `BLANK()` with `0`.

B — Run time categories (segmentation)

```
RunTimeCategory =
SWITCH(
    TRUE(),
    Movies[Run Time] < 90, "Short (<90)",
    Movies[Run Time] >= 90 && Movies[Run Time] < 120, "Medium (90-119)",
    Movies[Run Time] >= 120, "Long (>=120)",
    "Unknown"
)
```

Why column? Categories are static per row and used in slicers and visuals. Calculated column also reduces DAX complexity later.

---

4) Measures — DAX (aggregations, filter-aware)

General best practices:

- Use `VAR` to store intermediate results (readability + performance).
- Use `DIVIDE(n, d)` to avoid divide-by-zero.
- Respect filter context: measures evaluate in the visual's filter context automatically.
- Use `ALL` / `ALLSELECTED` / `ALLEXCEPT` to modify filter context intentionally.

Below are recommended measures with explanations.

4.1 Total Box Office

Simple aggregated measure:

```
Total Box Office = SUM( 'Movies'[Box Office] )
```

Test: Place in card → shows overall box office. Place in table by `Genre` to get per-genre totals (filter context in action).

4.2 Average Budget

Two ways — **per-movie average** or **weighted average** (if you want overall budget per movie vs weighted by box office). Typically `AVERAGE(Movies[Budget])` is per-movie average.

```
Average Budget = AVERAGE( 'Movies'[Budget] )
```

Note: If you want average budget of visible movies only, this is correct. If you need to ignore filters, use `CALCULATE( AVERAGE(...), ALL('Movies') ).`

4.3 Average Margin — *two approaches*, choose by business need

Option A — Aggregate margin = (sum profit) / (sum budget) — gives overall profit margin across dataset (recommended for financial summary).

```
Aggregate Margin =
VAR _TotalProfit = SUM( 'Movies'[Box Office] ) - SUM( 'Movies'[Budget] )
VAR _TotalBudget = SUM( 'Movies'[Budget] )
RETURN
IF( _TotalBudget = 0, BLANK(), DIVIDE( _TotalProfit, _TotalBudget ) )
```

Option B — Average of per-movie margins — each movie contributes equally:

```
Avg Per-Movie Margin =
AVERAGEX(
    'Movies',
    DIVIDE( 'Movies'[Box Office] - 'Movies'[Budget], 'Movies'[Budget] )
)
```

Difference: Option A weights by budget (big-budget movies influence result more). Option B gives simple mean of per-movie margins.

4.4 Total Movies with Oscars

Count rows with `Oscar Wins > 0`. Keep filter context (respect slicers) by using `ALLSELECTED` if needed to respect report selection.

```
Movies with Oscars =
CALCULATE(
    COUNTROWS( 'Movies' ),
    FILTER( ALLSELECTED('Movies'), 'Movies'[Oscar Wins] > 0 )
)
```

Explanation: `ALLSELECTED` ensures the count respects slicers on the page but ignores row-level filters from a visual if needed. If you want strict visual-level behavior, use `COUNTROWS( FILTER( 'Movies', 'Movies'[Oscar Wins] > 0 ) )`.

4.5 Top Genre by Box Office (returns genre name(s) as text)

This measure returns the top genre(s) by total box office in the current filter context.

```
Top Genre by Box Office =
VAR GenreTable =
    ADDCOLUMNS(
        SUMMARIZE( 'Movies', 'Movies'[Genre] ),
        "TotalBO", CALCULATE( SUM( 'Movies'[Box Office] ) )
    )
VAR TopRows = TOPN( 1, GenreTable, [TotalBO], DESC )
RETURN
CONCATENATEX( TopRows, 'Movies'[Genre], ", " )
```

Notes:

- `SUMMARIZE` + `ADDCOLUMNS` builds totals by genre.
- `TOPN(1, ...)` picks the top row; `CONCATENATEX` returns the genre text (handles ties if `TOPN` returns multiple).
- Place this in a card to show top genre overall. If filtered by Country, it will return top genre in that country due to filter context.

4.6 Year-over-Year (YoY) Box Office Growth

Time intelligence requires the `DateTable` and relationship. Use `SAMEPERIODLASTYEAR` or `DATEADD`.

```
YoY Box Office Growth =
VAR Current = CALCULATE( SUM( 'Movies'[Box Office] ) )
VAR Previous = CALCULATE( SUM( 'Movies'[Box Office] ), SAMEPERIODLASTYEAR(
'DateTable'[Date] ) )
RETURN
IF( ISBLANK(Previous) || Previous = 0, BLANK(), DIVIDE( Current - Previous,
Previous ) )
```

Explanation & testing:

- Put `DateTable[Year]` on x-axis of a line chart and this measure as KPI — it will compute growth year-to-year.
- If your date table is not contiguous or not marked, `SAMEPERIODLASTYEAR` may return blank — use `DATEADD('DateTable'[Date], -1, YEAR)` as alternative.

4.7 Average Nominations per Director

We want the average number of nominations per director (i.e., for each director compute avg nominations across their movies, then average across directors), or simply average nominations across movies grouped by director depending on need.

Measure — average nominations per director (average of directors' averages):

```
Avg Nominations per Director =
AVERAGEX(
    VALUES( 'Movies'[Director] ),
    CALCULATE( AVERAGE( 'Movies'[Nominations] ) )
)
```

Explanation:

- `VALUES('Movies'[Director])` iterates each distinct director in the current filter context; `CALCULATE(AVERAGE(...))` computes that director's average nominations; `AVERAGEX` averages these director-level averages.

Alternative (total nominations / number of directors):

```
Avg Nominations per Director (alt) =
VAR TotalNoms = SUM( 'Movies'[Nominations] )
VAR DirectorsCount = DISTINCTCOUNT( 'Movies'[Director] )
RETURN DIVIDE( TotalNoms, DirectorsCount )
```

Difference: the alt measure weights directors by total nominations; choose according to business meaning.

---

5) Visualization mapping & design advice

Page 1 — Overview

- **Cards:** place `Total Box Office`, `Aggregate Margin` (format as %), `Movies with Oscars`.
- **Stacked bar:** Axis = `Movies[Genre]`, Legend = `Movies[Certificate]`, Value = `Total Box Office`.
- **Line chart:** Axis = `DateTable[Year]` or `DateTable[Date]`, Value = `Total Box Office`. Add `YoY Box Office Growth` as KPI or secondary card.
- **Slicers:** `Movies[Country]`, `DateTable[Date]` (between slider).
- **KPI visual:** `YoY Box Office Growth` with target = 0 (green if >0, red if <0).

Page 2 — Director Analysis

- **Treemap:** Group = `Movies[Director]`, Values = `SUM(Movies[Budget])`, Color = `SUM(Movies[Oscar Wins])`.
- **Table:** columns: Director, `Total Nominations` (SUM), `Total Oscars` (SUM), `Avg Nominations per Director` (measure).
- **Slicer:** `Movies[Genre]`.
- **Donut chart:** `RunTimeCategory` distribution filtered by Director.

Page 3 — Genre & Country Insights

- **Matrix:** Rows = `Movies[Genre]`, Columns = `Movies[Country]`, Values = `Total Box Office`. Use conditional formatting → color scale on `Total Box Office`.
- **Pie chart:** `Certificate` share using `Total Box Office`.
- **Word cloud:** Genres (requires marketplace visual).
- **Slicer:** `RunTimeCategory`.

---

6) Filter context & common DAX patterns (explanations + examples)

Row context vs Filter context

- Row context: iterating over rows (calculated columns, `SUMX` row context inside iterators).
- Filter context: filters applied by visuals, slicers, or `CALCULATE`.

CALCULATE: changes filter context. Example: `CALCULATE( SUM(Movies[Box Office]), 'Movies'[Genre]="Action" )` forces genre to Action regardless of visual selection.

ALL / REMOVE filters:

- `CALCULATE( SUM(Movies[Box Office]), ALL(Movies) )` returns total ignoring all filters.
- `CALCULATE( SUM(Movies[Box Office]), ALL( Movies[Genre] ) )` ignores genre filter but respects others.

Percent of Total example:

```
% of Box Office by Genre =
DIVIDE(
    SUM( 'Movies'[Box Office] ),
    CALCULATE( SUM( 'Movies'[Box Office] ), ALL( 'Movies'[Genre] ) )
)
```

Placed in a matrix grouped by `Genre`, this shows each genre's share.

---

7) Edge cases & debugging tips

1. **Blanks and zeros:** Always use `DIVIDE` or check `IF(_den=0,BLANK(),...)`.
2. **Date table issues:** If `SAMEPERIODLASTYEAR` returns blank, verify:
   - Date table is marked as Date Table.
   - Relationship exists and is set correctly (Movies → DateTable).
   - DateTable covers full date range.
3. **Data types:** Numeric values stored as text will produce blanks in SUM. Fix in Power Query.
4. **Duplicated rows:** Verify early. Aggregations will over-count if duplicates exist.
5. **Performance:** Avoid `SUMX` over whole table when `SUM` will do. Use `VAR` to store repeated expressions.
6. **Validation:** Create quick table visuals with raw columns and measures to validate expected numbers (e.g., one row per movie with `Profit` shows correct arithmetic).

---

8) Sample testing checklist (do these to confirm correctness)

- ☐ Card: Total Box Office matches expected grand total (manual sum or Excel check).
- ☐ Table: Show `Title`, `Budget`, `Box Office`, `Profit` for first 10 rows — check calculations.
- ☐ Matrix: Genre × Country totals sum to overall `Total Box Office`.
- ☐ Time line: Line chart of `Total Box Office` by Year shows consistent trend; YoY growth matches manual calc for a sample year.
- ☐ Top Genre: Card shows expected genre for known dataset subset.
- ☐ Director page: Treemap sizes reflect budgets and colors correspond to Oscar wins.

---

9) Performance & modeling best practices (quick)

- Model as a star schema: one fact table (`Movies`) and small dimension tables (e.g., `Directors`, `Genres`, `Certificates`, `Countries`) if appropriate.
- Remove unused columns; hide columns not needed in reporting.
- Use numeric surrogate keys where possible to reduce cardinality.
- Use `VAR` in measures to avoid recomputation.
- Prefer measures over calculated columns for aggregations that must be dynamic; use calculated columns for row-level metadata (categories).
- Use Power Query for data cleaning (imputation, type conversion) — do not use DAX for heavy data transformations.

---

10) Deliverables you will write in practice (when you implement)

- Calculated columns: `Profit`, `RunTimeCategory`.

- Measures: `Total Box Office`, `Average Budget`, `Aggregate Margin`, `Avg Per-Movie Margin` (optional), `Movies with Oscars`, `Top Genre by Box Office`, `YoY Box Office Growth`, `Avg Nominations per Director`, `% of Box Office by Genre` (optional).
- Visual pages: Overview, Director Analysis, Genre & Country Insights with slicers and conditional formatting.

---

11) Final notes — clear guidance to avoid mistakes

- **Fix missing numeric values before modeling.** Don't attempt to "hide" missing budgets with measures — fill or flag them in Power Query.
- **Mark Date table** — most time-intelligence issues come from forgetting this step.
- **Decide definitions early** — e.g., choose whether "Average Margin" is aggregated margin or per-movie average; write both measures and label them clearly in the report.
- **Use concise, descriptive measure names** so reviewers/testers know what each number represents.